ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5 Дисциплина: Архитектура ЭВМ

Тема: Основы работы с Midnight Commander. Структура программы на языке ассемблера NASM. Системные вызовы.

Студент: А н ь с я о я н ь

Группа: НПИ-01-25

Студенческий билет: 1 1 3 2 2 5 8 1 9 1

Преподаватель: Демидова А. В.

1. Цель работы Приобретение практических навыков работы в Midnight Commander. Освоение инструкций языка ассемблера `mov` и `int`.

2. Выполнение работы 2.1. Подготовка рабочего пространства Я открыл Midnight Commander и создал каталог для пятой лабораторной работы.

- **Команда:** `mc`
- **Действие:** С помощью **F7** создал каталог `~/work/arch-pc/lab05`.
- **Команда:** `touch lab5-1.asm`
- **Результат:** Рабочий каталог настроен.

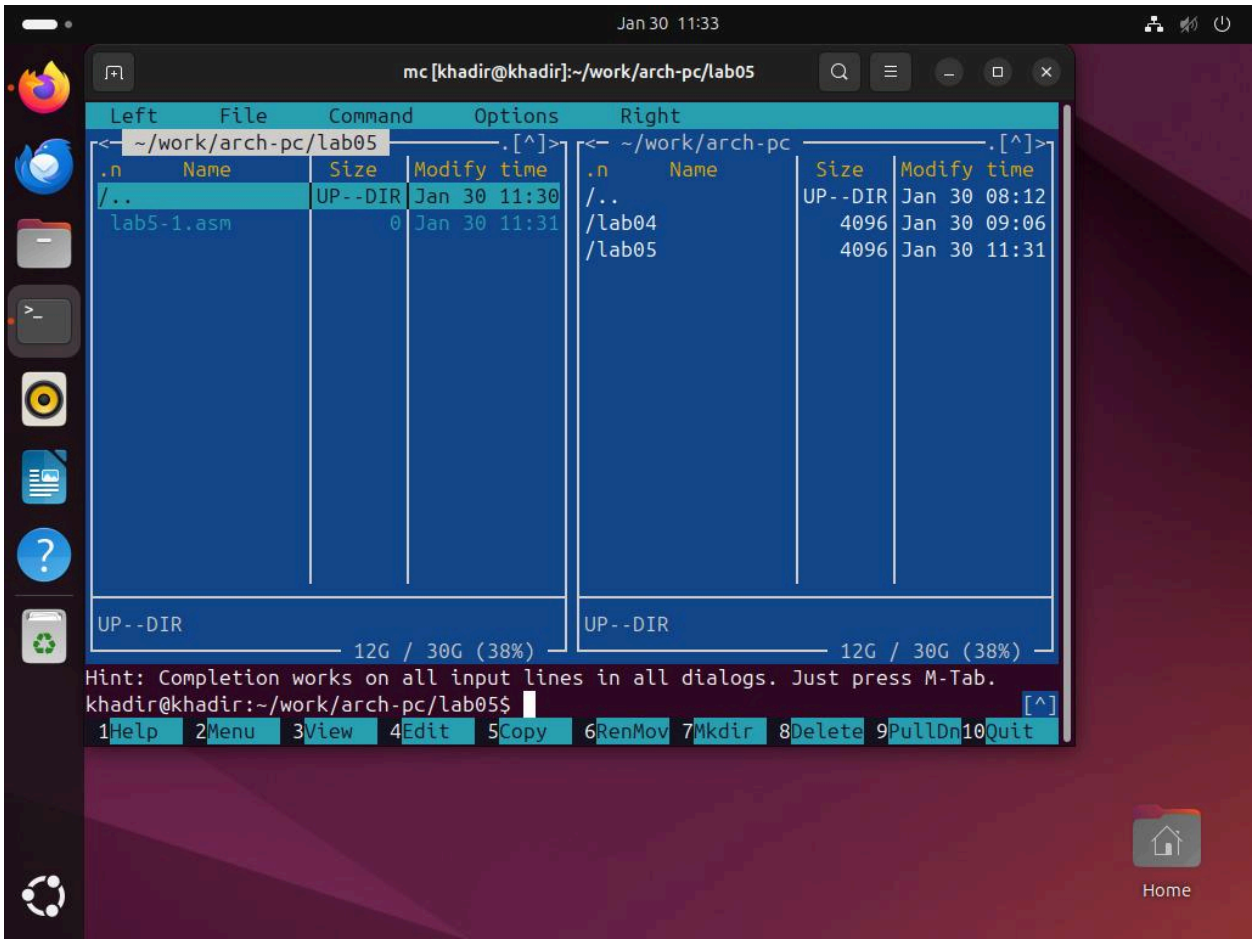

Рис. 5.1. Интерфейс Midnight Commander и создание папки.

2.2. Написание программы ввода-вывода (Листинг 5.1) Я открыл файл в редакторе (**F4**) и ввел текст программы. Программа использует системные вызовы для вывода сообщения и чтения строки.

- **Трансляция:** `nasm -f elf lab5-1.asm -o lab5-1.o`
- **Компоновка:** `i686-linux-gnu-ld -m elf_i386 lab5-1.o -o lab5-1`
- **Запуск:** `qemu-i386 ./lab5-1`
- **Результат:** Программа успешно скомпилирована и запущена через эмулятор.

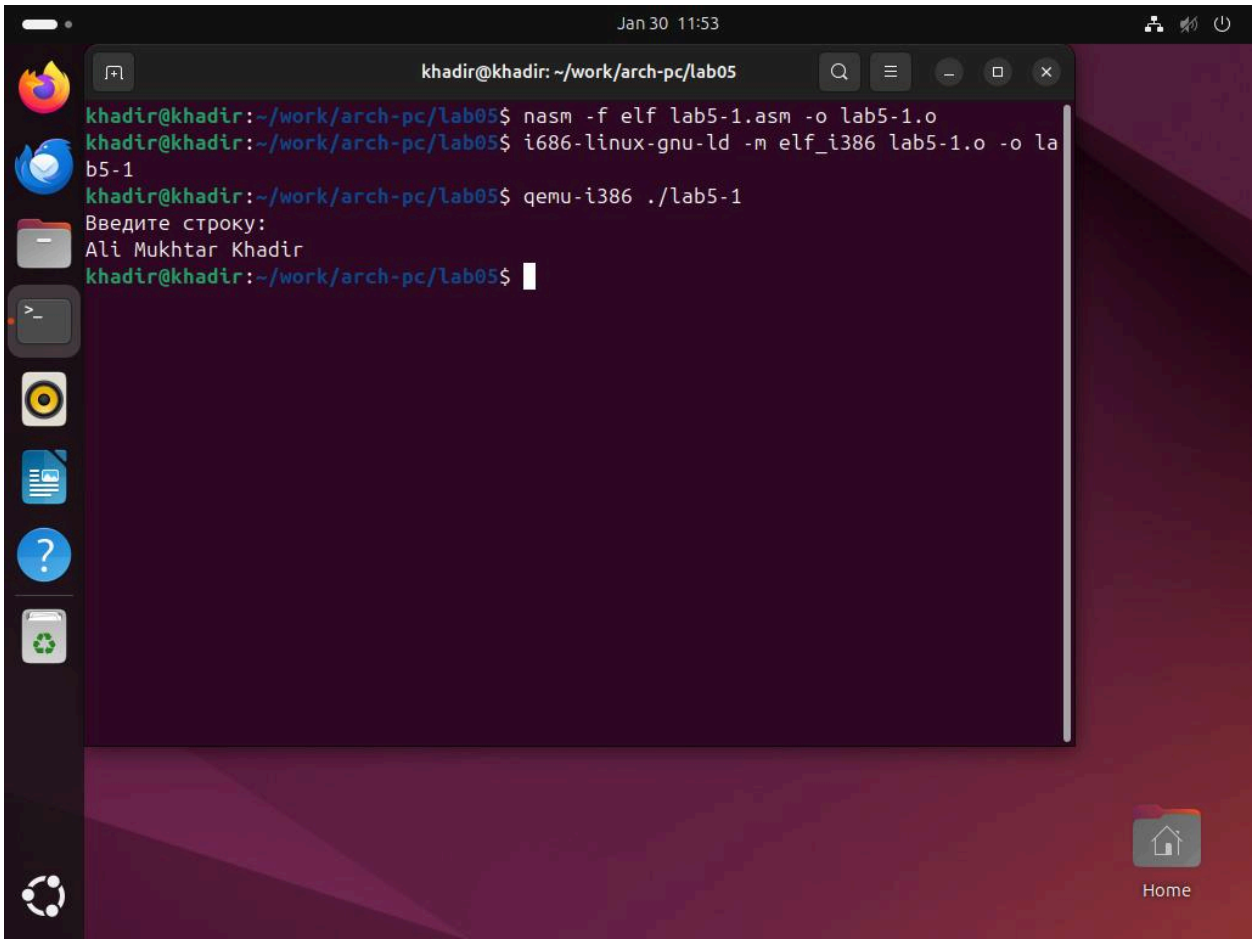

Рис. 5.2. Компиляция, компоновка и запуск первой программы lab5-1.asm.

2.3. Подключение внешней библиотеки `in_out.asm` Я скопировал файл библиотеки в рабочий каталог и изменил программу для использования упрощенных функций.

- **Действие:** Копирование `in_out.asm` через **F5**. Создание копии `lab5-2.asm` через **F6**.
- **Действие:** В коде использовал `%include 'in_out.asm'` и заменил вызовы `int 80h` на `call sprintLF`, `call sread` и `call quit`.
- **Результат:** Код стал более читаемым, программа работает корректно.

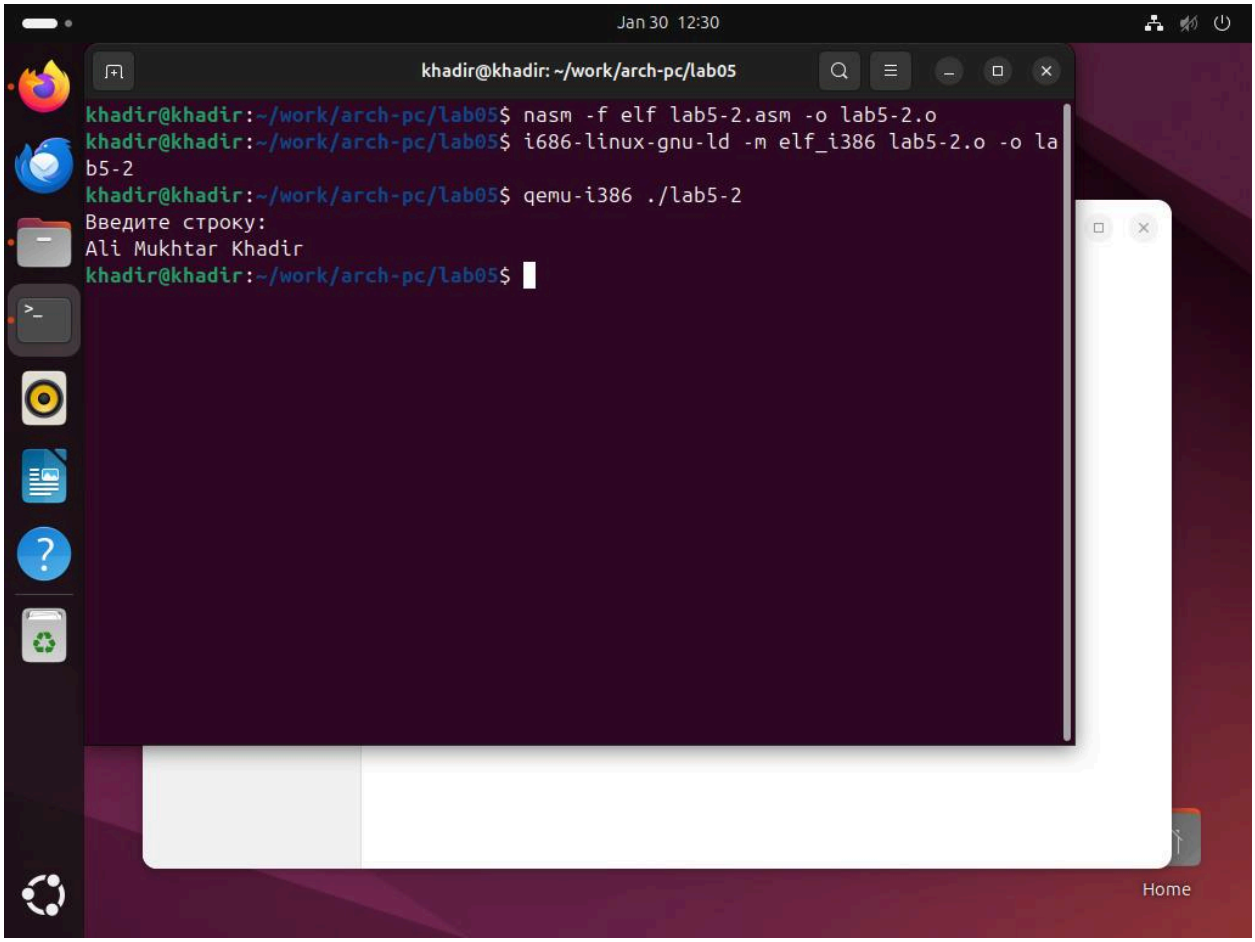

Рис. 5.3. Работа программы с использованием внешней библиотеки.

3. Самостоятельная работа

Я модифицировал программу так, чтобы она не только принимала ввод, но и выводила его обратно в терминал.

Действие: Создание файла `lab5-task.asm` и добавление инструкций `mov eax, buf1` и `call sprint` после функции чтения.

Действие: Трансляция и компоновка файла: `nasm -f elf lab5-task.asm -o lab5-task.o` `i686-linux-gnu-ld -m elf_i386 lab5-task.o -o lab5-task`

Результат: Программа выводит приглашение к вводу, считывает строку и выводит её на экран.

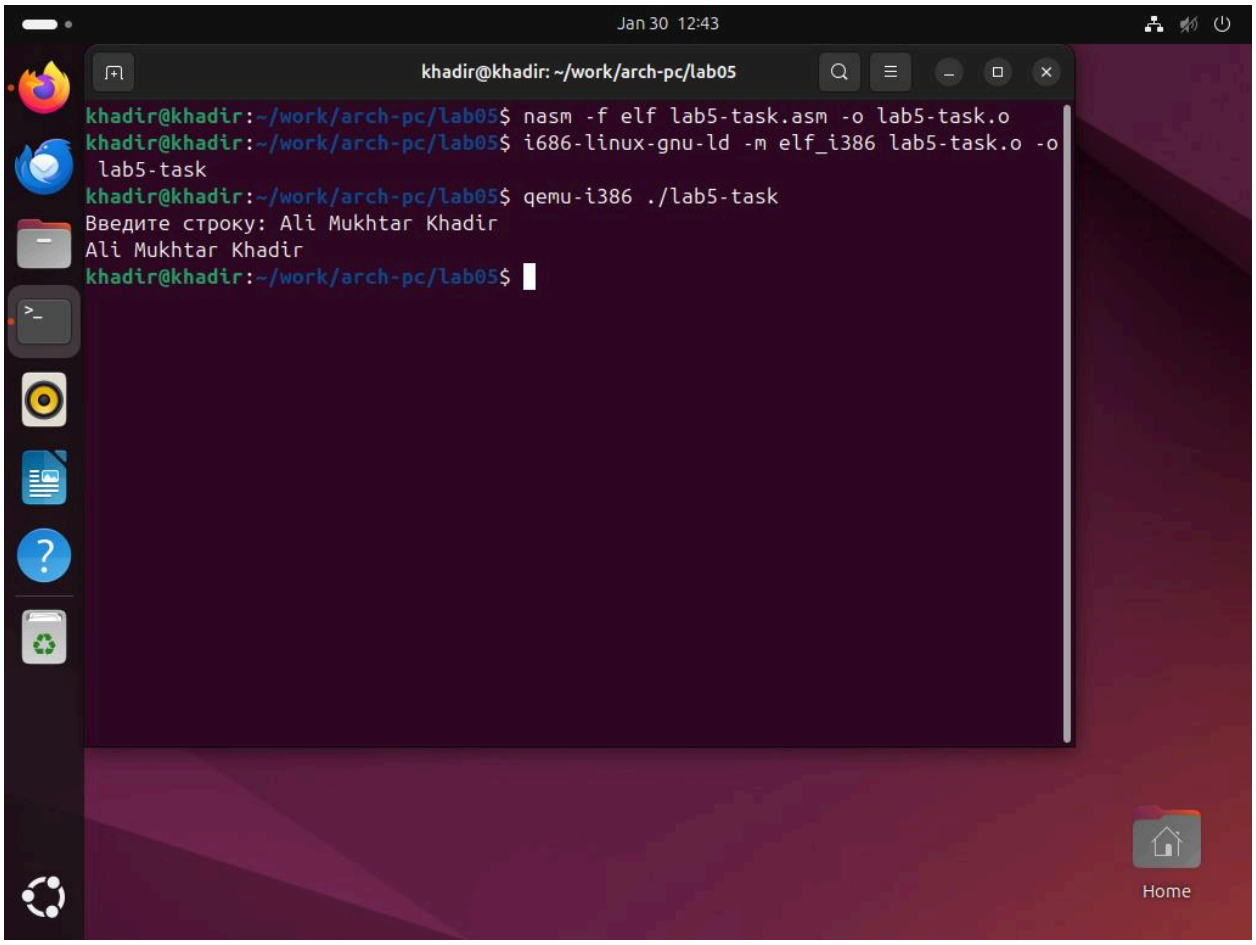

Рис. 5.4. Результат работы программы, выводящей введенную строку.

5.6. Ответы на контрольные вопросы

1. Каково назначение `mc`? Это программа для работы с файлами. В ней есть две панели, что удобно для копирования и поиска папок без ввода команд в терминале.

2. Какие операции с файлами можно выполнить в `bash` и в `mc`? Почти все. Например:

- **Копирование:** команда `cp` или кнопка **F5**.
- **Переименование:** команда `mv` или кнопка **F6**.
- **Удаление:** команда `rm` или кнопка **F8**.
- **Создание папки:** команда `mkdir` или кнопка **F7**.

3. Какова структура программы на языке ассемблера NASM? Обычно программа делится на три части:

- **`SECTION .data`** — для текста и готовых чисел.
- **`SECTION .bss`** — для пустого места под данные.
- **`SECTION .text`** — для самого кода программы.

4. Для чего используются секции `bss` и `data`?

- **`.data`** нужна для данных, которые мы знаем заранее (например, готовые сообщения).
- **`.bss`** нужна для данных, которые появятся только во время работы программы (например, то, что введет пользователь).

5. Для чего используются `db`, `dw`, `dd`, `dq` и `dt`? Это команды для выделения памяти разного размера:

- `db` — 1 байт.
- `dw` — 2 байта.
- `dd` — 4 байта.
- `dq` — 8 байт.
- `dt` — 10 байт.

6. Что делает команда `mov eax, esi`? Она берет значение из регистра `esi` и копирует его в регистр `eax`. В `esi` значение остается таким же.

7. Для чего используется `int 80h`? Это вызов системы. Эта команда говорит ядру Linux выполнить действие: например, вывести текст на экран или закончить программу.

4. Вывод

В ходе выполнения лабораторной работы №5 я изучил механизмы ввода и вывода данных на языке ассемблера NASM. Я научился использовать как прямые системные вызовы через прерывание `int 80h`, так и упрощенные функции из внешней библиотеки `in_out.asm`. Мною была успешно решена задача по модификации программы для вывода текста из буфера памяти. Полученные навыки работы с регистрами и секциями `.data` и `.bss` позволяют создавать более сложные программы и эффективно управлять памятью.